DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 2: GUI-Based Algorithm Implementation

PowerWise

Smart Electricity Usage Optimizer

0/1 Knapsack using Dynamic Programming

Item	Details
Student Name	Rosh K Lijo
Roll Number	26
Admission Number	16611
Algorithm	0/1 Knapsack (Dynamic Programming)
Language / GUI	Python 3 / Tkinter
Main source files	main.py, gui/app.py, algorithms/knapsack.py

Submitted as part of the Design and Analysis of Algorithms assignment.

1. Problem Statement

Electricity usage in homes and offices can be managed by choosing a suitable combination of appliances within a fixed power limit. Each appliance consumes a certain amount of electrical power (in watts) and has a priority value representing its importance. When the available power capacity is limited, the system must select the most valuable combination of appliances without exceeding that capacity.

The objective of PowerWise is to help a user choose appliances under a specified power limit. The application uses the 0/1 Knapsack algorithm with Dynamic Programming to find the combination with the maximum total priority. Each appliance can either be selected once or not selected; it cannot be selected multiple times.

Objectives

• Apply the 0/1 Knapsack problem to a practical electricity-management scenario.
• Provide a graphical interface for entering appliance details and power capacity.
• Allow users to add, edit and delete appliance records.
• Display the selected appliances, total power, unused capacity and total priority.
• Visualize the Dynamic Programming table step by step.
• Explain the algorithm and analyse its time and space complexity.
• Test normal inputs, boundary cases and invalid inputs.

2. Algorithm

2.1 Why Dynamic Programming

The 0/1 Knapsack problem has two important properties: optimal substructure and overlapping subproblems. An optimal solution for the first i appliances and a capacity w can be constructed using solutions for smaller sets of appliances and smaller capacities. Many of these smaller subproblems occur repeatedly. Dynamic Programming stores their answers in a table, avoiding repeated calculations and producing an optimal result.

2.2 State Definition and Recurrence

Let dp[i][w] represent the maximum total priority that can be obtained using the first i appliances without exceeding power capacity w. For each appliance, there are two choices: exclude it, or include it if its power requirement fits.

Base case: dp[0][w] = 0 for every capacity w, and dp[i][0] = 0 for every appliance count i.

Recurrence:

```
If power[i] > w:
dp[i][w] = dp[i-1][w]

Otherwise:
dp[i][w] = max(dp[i-1][w], priority[i] + dp[i-1][w - power[i]])
```

The first value represents skipping the current appliance. The second represents selecting it and adding its priority to the best result for the remaining capacity.

2.3 Steps of the Algorithm

1. Read the list of appliances and the maximum power capacity.
2. Validate the appliance names, power values, priority values and capacity.
3. Create a DP table with (n + 1) rows and (W + 1) columns, initialized to zero.
4. Process appliances one by one.
5. For every capacity from 0 to W, compare the value obtained by excluding the appliance with the value obtained by including it, if it fits.
6. Store the larger value in the DP table.
7. Trace backwards through the table to identify the selected appliances.
8. Calculate total power, remaining capacity and total priority, and display the results.

2.4 Pseudocode

```
ZERO-ONE-KNAPSACK(appliances, W)
n = number of appliances
create dp[0..n][0..W], initialized to 0
for i = 1 to n
for w = 0 to W
dp[i][w] = dp[i-1][w] // exclude appliance i
if power[i] <= w
include = priority[i] + dp[i-1][w-power[i]]
dp[i][w] = max(dp[i][w], include)
trace back through dp to find selected appliances
return selected appliances and totals
```

2.5 Why it is a 0/1 Knapsack

The application uses each appliance at most once. The recurrence refers to the previous row, dp[i-1], when an appliance is included. This prevents the same appliance from being selected repeatedly. Therefore, the problem is 0/1 Knapsack and not the unbounded knapsack problem.

3. Complexity Analysis

Let n be the number of appliances and W be the maximum power capacity, measured in watts.

Operation	Time Complexity	Space Complexity	Reason
DP table construction	O(n × W)	O(n × W)	There are n × W states; each state performs a constant-time comparison.
Traceback / selection	O(n)	O(n) for selected list	At most one decision is made for each appliance.
Input validation	O(n)	O(1) auxiliary	Each appliance record is checked once.

Overall time complexity: O(n × W).
Overall space complexity: O(n × W), because the complete DP table is stored for result reconstruction and step visualization.

The algorithm is pseudo-polynomial because its running time depends on the numeric capacity W, rather than only on the number of bits needed to represent W. For a larger capacity, the DP table becomes larger.

Best case: when there are no appliances or the capacity is zero, the resulting priority is zero and the table contains only base-case values (although table initialization still takes space).
Worst case: when there are n appliances and a large capacity W, all table states are evaluated.

4. System Design

File / Component	Purpose
main.py	Starts the application by calling run_app().
gui/app.py	Creates the Tkinter graphical interface, manages appliance records, accepts capacity, displays results and provides the DP-step viewer.
algorithms/knapsack.py	Contains the 0/1 Knapsack algorithm, input validation, DP table construction and traceback of selected appliances.
tests/test_knapsack.py	Contains automated unit tests for algorithm correctness and invalid or boundary inputs.
assets/sample_appliances.csv	Stores sample appliance data used for demonstration.

GUI Features

• Appliance management: add, edit and delete appliance entries.
• Capacity input: enter the maximum permitted power in watts.
• Optimization: run the 0/1 Knapsack algorithm and display the best appliance combination.
• Result summary: show selected appliances, total power used, remaining capacity and total priority score.
• DP visualization: open a separate window and move through the algorithm steps using Previous Step and Next Step controls.
• Input validation: reject invalid capacity, appliance names, power values and priority values.

Input and Output

Inputs include appliance name, power consumption in watts, priority value and maximum power capacity. Outputs include the selected and unselected appliances, total power used, remaining capacity, maximum priority score and the DP table used to calculate the optimal solution.

5. GUI Screenshots

Insert screenshots captured from the completed PowerWise application in the spaces below. Use actual screenshots from the running program; do not use screenshots from another project.

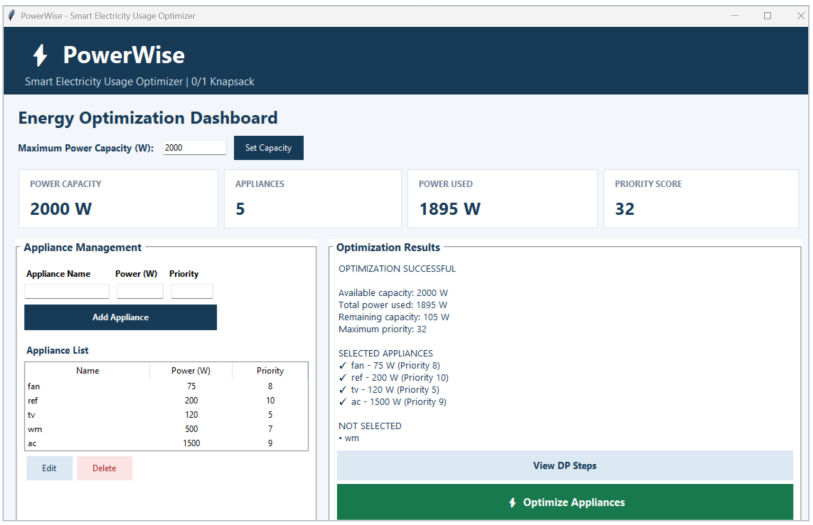


Figure 1: PowerWise main window.

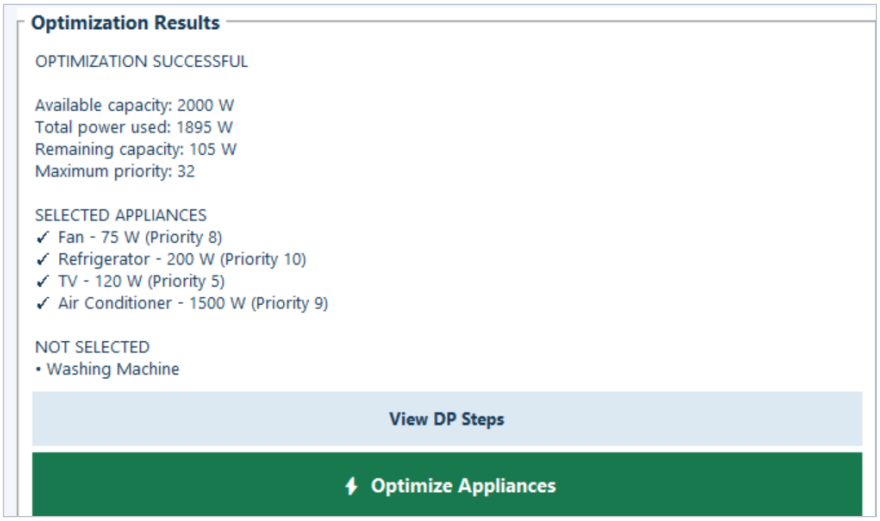


Figure 2: Optimization result for the sample input.

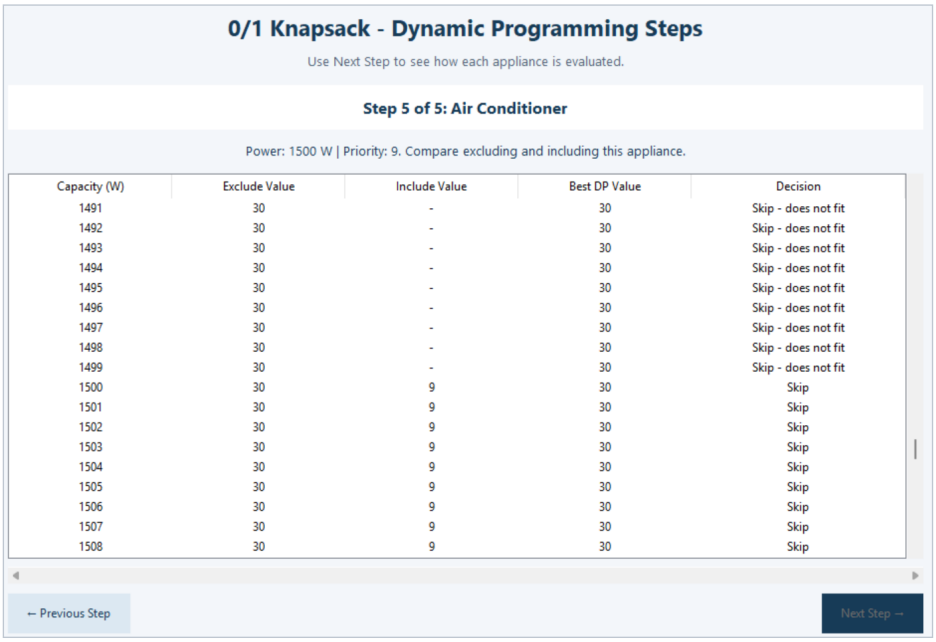


Figure 3: DP table visualization with Previous Step and Next Step controls.

6. Testing and Results

The project includes automated unit tests in tests/test_knapsack.py. The test suite contains eight test cases covering normal operation and important boundary or invalid-input conditions. Run the tests from the project folder using the command shown in Section 8. Record the actual test output and update the result column if any test fails.

#	Test Case	Expected Behaviour	Result
1	Normal input	Returns an optimal selection within capacity.	To be verified
2	Zero capacity	Selects no appliances; total power and priority are zero.	To be verified
3	Exact capacity	Allows a combination whose total power equals the capacity.	To be verified
4	Insufficient capacity	Does not select an appliance that exceeds capacity.	To be verified
5	Empty appliance list	Returns an empty selection and zero totals.	To be verified
6	Invalid capacity	Rejects negative or invalid capacity values.	To be verified
7	Invalid power value	Rejects invalid appliance power values.	To be verified
8	Invalid priority value	Rejects invalid appliance priority values.	To be verified

6.1 Sample Run

The following sample input was used to demonstrate the application. The maximum power capacity is 2000 W.

Appliance	Power (W)	Priority
Fan	75	8
Refrigerator	200	10
Television	120	5
Washing Machine	500	7
Air Conditioner	1500	9

Capacity: 2000 W

Output	Value
Selected appliances	Fan, Refrigerator, Television, Air Conditioner
Total power used	1895 W
Remaining capacity	105 W
Total priority score	32

The selected combination uses 1895 W, which does not exceed the 2000 W capacity. Its total priority is 32. The Washing Machine is not selected in this optimal combination. This sample demonstrates how the algorithm balances appliance power consumption and priority.

6.2 Observations

The DP algorithm evaluates both possibilities for every appliance: including it when it fits, or excluding it. The final result is an optimal combination under the given capacity, rather than a selection based only on the highest individual priority or lowest power consumption. The DP-step window makes the intermediate table values available for inspection.

7. Conclusion

PowerWise demonstrates the use of the 0/1 Knapsack problem and Dynamic Programming in a practical electricity-usage optimization scenario. The application allows users to manage appliance details, set a power capacity and obtain an optimal combination of appliances based on their priority values. It also provides a step-by-step view of the DP table, helping users understand how the algorithm reaches its result.

The algorithm has O(n × W) time complexity and O(n × W) space complexity when the full table is stored. The sample run selects Fan, Refrigerator, Television and Air Conditioner, using 1895 W out of 2000 W and achieving a priority score of 32. Possible future enhancements include estimating electricity cost using usage hours and tariff rates, saving appliance records permanently, and comparing different capacity settings.

8. How to Run

Open PowerShell in the project folder:

```
cd C:\Users\YOGA\Documents\PowerWise
```

Activate the virtual environment if it is not already active:

```
.\.venv\Scripts\Activate.ps1
```

Run the application:

```
python main.py
```

Run the unit tests:

```
python -m unittest discover -s tests -v
```

Requirements

Python 3 with Tkinter is required. The project uses the Python standard library for the graphical interface and algorithm. Any additional packages required by the local project environment should be installed from its project dependency file, if present.

Note: Before submission, fill in the student details on the cover page, insert the actual screenshots, and replace “To be verified” with the test results from your own test run.